

Python Functions – Complete Notes

What is a Function?

A function is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, we can write it once inside a function and call it whenever needed.

Advantages of Functions

- Code reusability
- Reduces code duplication
- Improves readability
- Easier debugging
- Better code organization
- Makes programs modular

Syntax of a Function

```
def function_name():  
    # Function body  
    print("Hello Python")
```

```
function_name()
```

Output

Hello Python

Creating and Calling a Function

```
def greet():  
    print("Welcome to Python Course")
```

```
greet()
```

Output

Welcome to Python Course

Function with Parameters

Parameters are variables that receive values when a function is called.

```
def greet(name):  
    print("Hello", name)  
  
greet("Ravi")
```

Output

Hello Ravi

Function with Multiple Parameters

```
def student_details(name, course):  
    print("Name:", name)  
    print("Course:", course)  
  
student_details("Anu", "Python")
```

Output

Name: Anu
Course: Python

Function with Return Statement

The return keyword sends a result back to the caller.

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

Output

30

Difference Between print() and return

Using print()

```
def add(a, b):  
    print(a + b)
```

```
add(10, 20)
```

Output

30

Using return()

```
def add(a, b):  
    return a + b
```

```
result = add(10, 20)  
print(result)
```

Output

30

Key Difference

print()	return
Displays output	Sends value back
Cannot reuse result easily	Result can be stored
Used for displaying	Used for processing

Types of Functions

1. Built-in Functions

Python provides predefined functions.

Examples:

```
print()  
len()  
max()  
min()  
sum()
```

```
type()
input()
```

Example:

```
numbers = [10, 20, 30]

print(len(numbers))
print(max(numbers))
```

Output

```
3
30
```

2. User Defined Functions

Functions created by programmers.

```
def welcome():
    print("Welcome")

welcome()
```

Function with No Arguments and No Return Value

```
def show_message():
    print("Learning Python")

show_message()
```

Output

```
Learning Python
```

Function with Arguments and No Return Value

```
def multiply(a, b):
    print(a * b)

multiply(4, 5)
```

Output

20

Function with Arguments and Return Value

```
def multiply(a, b):  
    return a * b  
  
result = multiply(4, 5)  
print(result)
```

Output

20

Function with No Arguments and Return Value

```
def get_number():  
    return 100  
  
print(get_number())
```

Output

100

Positional Arguments

Arguments are passed in the same order as parameters.

```
def student(name, age):  
    print(name, age)  
  
student("Ravi", 21)
```

Output

Ravi 21

Keyword Arguments

Arguments are passed using parameter names.

```
def student(name, age):  
    print(name, age)  
  
student(age=21, name="Ravi")
```

Output

Ravi 21

Default Arguments

Default values are assigned to parameters.

```
def greet(name="Guest"):  
    print("Hello", name)  
  
greet()  
greet("Ravi")
```

Output

Hello Guest
Hello Ravi

Variable Length Arguments (*args)

Used when the number of arguments is unknown.

```
def add_numbers(*args):  
    print(args)  
  
add_numbers(10, 20, 30, 40)
```

Output

(10, 20, 30, 40)

Example:

```
def add_numbers(*args):  
    return sum(args)
```

```
print(add_numbers(10, 20, 30))
```

Output

```
60
```

Keyword Variable Length Arguments (**kwargs)

Accepts multiple keyword arguments.

```
def student_info(**kwargs):  
    print(kwargs)  
  
student_info(name="Ravi", age=21)
```

Output

```
{'name': 'Ravi', 'age': 21}
```

Local Variables

Variables declared inside a function.

```
def demo():  
    x = 100  
    print(x)  
  
demo()
```

Output

```
100
```

Global Variables

Variables declared outside a function.

```
x = 100  
  
def demo():  
    print(x)  
  
demo()
```

Output

100

Global Keyword

```
count = 10

def update():
    global count
    count = 20

update()
print(count)
```

Output

20

Nested Functions

A function inside another function.

```
def outer():
    print("Outer Function")

    def inner():
        print("Inner Function")

    inner()

outer()
```

Output

Outer Function
Inner Function

Recursive Functions

A function calling itself.

```
def countdown(n):
    if n == 0:
        print("Done")
    else:
```

```
    print(n)
    countdown(n - 1)

countdown(5)
```

Output

```
5
4
3
2
1
Done
```

Lambda Functions

Anonymous one-line functions.

```
square = lambda x: x * x

print(square(5))
```

Output

```
25
```